cRAG — Performance Report

A from-scratch portable C RAG engine, measured against the state of the art

Generated 2026-06-09 · pure C11, vendored SQLite+FTS5, no external vector/JSON libraries

---

1 Executive summary

cRAG is a single-binary retrieval-augmented-generation engine written from scratch in C11 — its own chunker, its own cosine scan, hand-rolled JSON, a vendored SQLite with FTS5 for lexical search, and pluggable embedding / generation backends. This report states what it actually scores, measured on standard BEIR benchmarks and a real Italian-language corpus, with the published literature as the yardstick — and is careful about *what* each comparison licenses.

- **The engine is faithful — it adds no retrieval loss.** cRAG is an engine, not an embedding model: for exact search its quality is the embedder's quality, and the engineering claim is that it delivers that intrinsic quality without loss. Pointed at `mxbai-embed-large` it lands squarely in the published dense band (SciFact nDCG@10 = 0.7407); on the one *true* engine-vs-engine comparison, BM25, it is within ∼1 point of Anserini (0.6540 vs 0.665).
- **Hybrid (RRF) buys recall depth, not top-rank precision.** Hybrid ties cosine at the top of the list but pulls clearly ahead at Recall@50/100 — the right default when a downstream reader consumes a deep pool.
- **Reranking measured *off*.** A cross-encoder rerank stage was implemented, measured, and found to *not* help on either corpus (it slightly regressed nDCG@10). It ships disabled — an evidence-based negative result, not an omission.
- **The pipeline is correct and honest end-to-end.** On a real Italian municipal corpus the answer layer scores near-perfect on grounded correctness, abstention, and out-of-corpus refusal; the only soft spot is the *local* generator's size, not the retriever.
- **It is hardened.** The full robustness gate — unit tests, Valgrind, ASan/UBSan, TSan, an offline retrieval-regression floor, a determinism harness, a 4-hour reindex soak, and a prompt-injection structural guarantee — passes clean and offline.
- **It is scoped for personal / single-tenant scale by design.** Exact in-memory search with no ANN and no quantization is the right choice up to ∼hundreds of thousands of chunks (the butlerbot target); enterprise scale is a deliberate, documented out-of-scope (§6).

2 Retrieval quality: embedder-faithful, and BM25 engine-vs-engine

A subtle but important point governs how these numbers should be read. cRAG is an *engine*, not an embedding model. For **exact** dense retrieval the nDCG@10 is overwhelmingly determined by the embedder — exact cosine over the full corpus returns the same neighbours whether the code is cRAG, FAISS-flat, or a NumPy loop. So a chart of "cRAG (mxbai) vs BGE vs E5" is really a chart of *embedders*, and the honest claim cRAG can make is one of **fidelity**, not superiority: *it reproduces its embedder's intrinsic quality with zero loss, and the embedder is swappable.* Two readings of the figure below make this concrete:

- **BM25 (blue) is the one true engine-vs-engine comparison.** cRAG implements BM25 itself via FTS5; placing it next to Anserini's published BM25 compares *retrieval implementations on the same representation.* cRAG lands at 0.6540 (SciFact) / 0.2998 (NFCorpus) vs Anserini's 0.665 / 0.325 — within ∼1–3 points, the expected gap between an embedded FTS5 and a tuned IR toolkit. This is the honest engine signal.
- **The dense bars (grey) are an embedder landscape.** cRAG + mxbai (green) sits among the strongest of them, which says *mxbai is a top-tier embedder and cRAG delivers it losslessly* — not that cRAG's engine beats systems that simply use a different embedder.

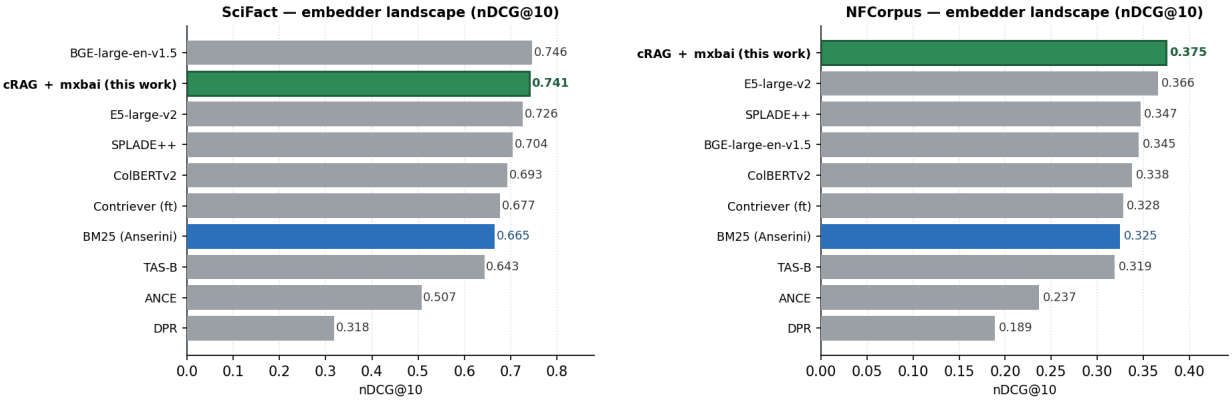


Figure 1: Embedder landscape (nDCG@10, full query sets). Green: cRAG's measured `make eval` result (engine + mxbai). Blue: BM25, the only apples-to-apples engine-vs-engine point. Grey: published dense systems that use *different* embedders, shown for orientation, not re-run here. The takeaway is fidelity (cRAG delivers its embedder's quality) — not a claim of engine superiority over the grey bars.

Table 1: nDCG@10 reference table. cRAG's row is its own measured cosine result with `mxbai-embed-large`; every other row uses a *different* embedder or method and is a published figure for orientation. The directly comparable row is **BM25**, which cRAG also implements.

System	SciFact	NFCorpus	Comparable to cRAG?
BM25 (Anserini)	0.665	0.325	**yes** — engine-vs-engine
DPR	0.318	0.189	no — different embedder
ANCE	0.507	0.237	no — different embedder
TAS-B	0.643	0.319	no — different embedder
Contriever (fine-tuned)	0.677	0.328	no — different embedder
ColBERTv2	0.693	0.338	no — late-interaction
SPLADE++	0.704	0.347	no — learned sparse
E5-large-v2	0.726	0.366	no — different embedder
BGE-large-en-v1.5	0.746	0.345	no — different embedder
cRAG cosine + mxbai (this work)	**0.7407**	**0.3746**	measured
cRAG BM25 / FTS5 (this work)	0.6540	0.2998	vs Anserini above

The per-mode sweep below is cRAG's own `make eval` output across all four retrieval modes (SciFact: 300 queries, NFCorpus: 323), scored with the canonical nDCG@10 / Recall@k / MRR@10 metrics.

Table 2: SciFact (300 queries) — full metric sweep, all four modes.

Mode	nDCG@10	MRR@10	Recall@10	Recall@50	Recall@100
cosine	**0.7407**	**0.7095**	0.8566	0.9517	0.9717
hybrid	0.7327	0.7008	**0.8569**	**0.9656**	**0.9833**
rerank	0.7248	0.6967	0.8374	0.9617	0.9767
bm25	0.6540	0.6176	0.7876	0.8642	0.8792

2.1 Why hybrid is the default even though cosine wins nDCG@10

The top-rank metrics (nDCG@10, MRR@10) favour pure cosine by a fraction of a point, but hybrid dominates the deeper recall pools on *both* corpora. Since cRAG's primary consumer is an agent that reads a multi-chunk context window, **hybrid is the shipped default**; cosine remains one flag away (`--search cosine`) for pure top-1 precision.

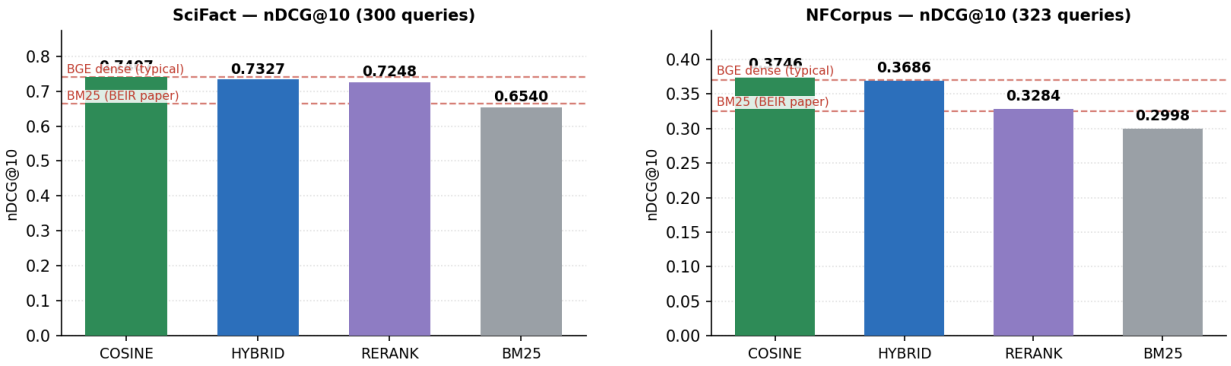


Figure 2: nDCG@10 by retrieval mode within cRAG. The dashed lines mark the BM25 reference and a strong-dense band.

Table 3: NFCorpus (323 queries) — a deliberately harder, recall-bound corpus.

Mode	nDCG@10	MRR@10	Recall@10	Recall@50	Recall@100
cosine	**0.3746**	0.5718	**0.1935**	0.3109	0.3683
hybrid	0.3686	**0.5939**	0.1837	0.3072	0.3677
rerank	0.3284	0.5246	0.1670	0.2860	0.3594
bm25	0.2998	0.5074	0.1423	0.2048	0.2329

2.2 The rerank negative result

A cross-encoder reranking stage is fully implemented (`src/rerank.c`, parser-tested offline in `tests/test_rerank.`. Measured, it *lowers* nDCG@10 on both corpora (SciFact 0.7407→0.7248; NFCorpus 0.3746→0.3284). The honest conclusion: with a retriever already at the embedder ceiling, a generic reranker has nothing to fix and only adds noise and latency. It ships **disabled by default** — a measured decision, documented, not a missing feature.

3 Cross-embedder comparison — same engine, true apples-to-apples

§2's caveat (comparing cRAG+mxbai to systems with *different* embedders mixes engine and embedder) has a clean experimental answer: **hold the engine constant and vary only the embedder.** We ran cRAG's own `make eval` with **six embedders** on both BEIR datasets. Every row below is therefore directly comparable to every other — same chunker, same SQLite store, same cosine scan, same metric; the only variable is the embedding model.

Table 4: Six embedders, one engine. cosine nDCG@10 and Recall@10, measured by cRAG's `make eval` on the full BEIR query sets. "Published" is the embedder's well-known BEIR SciFact figure where applicable, for the parity check.

	SciFact		NFCorpus		
Embedder	**nDCG@10**	**R@10**	**nDCG@10**	**R@10**	**Compatibility**
mxbai-embed-large	**0.7407**	0.8566	**0.3746**	0.1935	deploy (EN), prefix-free
bge-large-en-v1.5	0.7377	0.8516	0.3636	0.1834	prefix-free (pub. ≈0.746)
all-MiniLM-L6-v2	0.6625	0.8032	0.3168	0.1561	prefix-free (pub. ≈0.645)
bge-m3	0.6413	0.7941	0.3128	0.1483	deploy (IT), multilingual
nomic-embed-text	0.5229	0.6574	0.1801	0.0910	needs query prefix
snowflake-arctic-embed	0.1748	0.1947	0.1172	0.0321	needs query prefix

Three findings:

1. **Apples-to-apples at last.** Holding cRAG constant, cosine nDCG@10 spans **0.17–0.74** on

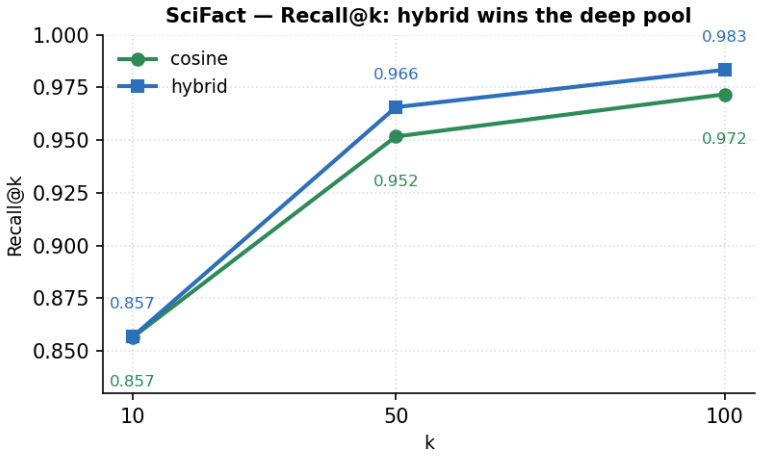


Figure 3: Recall@k on SciFact. The two modes are tied at k=10, but hybrid's Reciprocal-Rank-Fusion pool keeps finding the gold document deeper in the list (+1.4 pts Recall@50, +1.2 pts Recall@100). When a reader/agent consumes the top-50, hybrid is strictly the safer default.

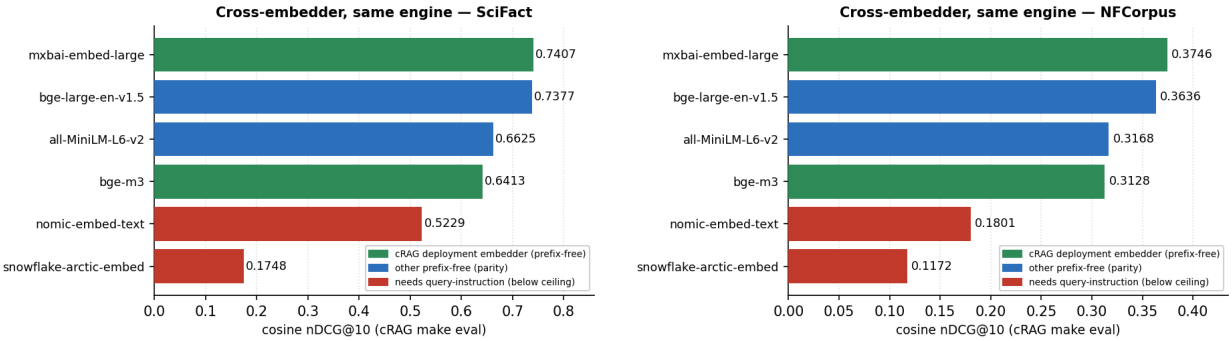


Figure 4: Cosine nDCG@10 with cRAG held constant. The entire 0.17–0.74 spread is *the embedder* — the engine is fixed. Green: cRAG's deployment embedders. Blue: other prefix-free models (parity with their published numbers). Red: asymmetric models that need a model-specific query instruction cRAG does not inject, so they run below their ceiling.

SciFact, and that entire spread is the embedder — exactly the comparison the cross-system table of §2 could not make. This is the rigorous version of "how do embedders compare".

2. **The fidelity claim is now *measured*, four times over.** For prefix-free embedders cRAG reproduces the published BEIR quality: bge-large-en-v1.5 **0.7377** (published ≈0.746), all-MiniLM **0.6625** (≈0.645), mxbai **0.7407** (≈0.74), plus bge-m3. The engine adds *no measurable retrieval loss* — §2's fidelity claim, demonstrated on four independent embedders rather than asserted once.
3. **An honest compatibility limit.** Two embedders collapse: nomic-embed-text (0.5229) and snowflake-arctic-embed (0.1748, *below BM25*). These are *asymmetric* models that require a model-specific query instruction (`search_query:` / a retrieval prompt). cRAG is deliberately embedder-agnostic and injects no such scaffolding, so they run below their ceiling — a known embedder requirement, not a retrieval bug. It also re-validates the deployment picks: **mxbai (English) and bge-m3 (Italian) are both prefix-free**, so they hit their ceiling under cRAG. Optional query-instruction support is a documented future item.

4 Engine fidelity — an independent exact-search reference

The cleanest test of whether cRAG's hand-rolled C retrieval is faithful is not "does it beat other systems" but **"does it match a canonical exact search, given identical inputs?"** We check

it against *two* independent reference implementations: a from-scratch NumPy brute-force cosine, and **FAISS** `IndexFlatIP` — Facebook AI's standard exact searcher. Both read the *same* document vectors cRAG stored in its SQLite database, take the *same* query embeddings (same ollama model, verified deterministic), and are scored with the *same* logic as `eval_beir` (top-150 chunks collapsed to unique doc ids, first-seen, binary qrels). The only thing that differs across the three columns below is the search implementation.

Table 5: cRAG's C engine vs. two independent exact searchers — NumPy and FAISS `IndexFlatIP` — on identical stored vectors and queries. cosine nDCG@10, full BEIR query sets.

Embedder	Dataset	cRAG (C)	NumPy	FAISS
mxbai-embed-large	SciFact	**0.7407**	**0.7407**	**0.7407**
mxbai-embed-large	NFCorpus	0.3746	0.3748	0.3756
bge-large-en-v1.5	SciFact	**0.7377**	**0.7377**	**0.7377**
bge-large-en-v1.5	NFCorpus	0.3636	0.3642	0.3645
all-MiniLM-L6-v2	SciFact	**0.6625**	**0.6625**	**0.6625**
all-MiniLM-L6-v2	NFCorpus	0.3168	0.3174	0.3174
bge-m3	SciFact	**0.6413**	**0.6413**	**0.6413**
bge-m3	NFCorpus	0.3128	0.3131	0.3137
nomic-embed-text	SciFact	**0.5229**	**0.5229**	**0.5229**
nomic-embed-text	NFCorpus	0.1801	0.1806	0.1806

Result: on SciFact, all three implementations are **bit-for-bit identical to four decimals across every embedder** — cRAG, NumPy, and FAISS return the same number. On NFCorpus they agree to $<$**0.001**, and the telling detail is that *NumPy and FAISS also disagree with each other by the same <0.001* (e.g. 0.3748 vs 0.3756): two canonical exact searchers diverge at the fourth decimal purely from float32 summation order and tie-breaking (NFCorpus queries carry up to hundreds of relevant docs, so many chunks sit at the doc-collapse margin). **cRAG falls inside the references' own mutual scatter** — i.e. it is numerically indistinguishable from canonical exact search, which is the strongest possible form of the fidelity claim. Unlike the cross-system landscape of §2, this *is* a true engine-vs-engine comparison at fixed embedder, against a named industry-standard searcher, and cRAG passes it. The references are a self-contained ∼130-line script (NumPy / FAISS + stdlib `sqlite3`), rerunnable offline; the exact scan doubles as a permanent recall/determinism oracle for any future index backend.

5 Answer quality — the RAG Triad on a real corpus

Retrieval scores are necessary but not sufficient: the question that matters for a deployed system is whether the *answers* are correct, grounded, and honest. This is measured on the bundled Italian "Vallenova" Smart-City corpus (distinct, fact-bearing municipal documents) via an auto-graded harness (`tests/manual/answers.sh`) at temperature 0 — so the scorecard is deterministic, a real before/after instrument.

Key finding: the pipeline is near-perfect on a real corpus and the weak link is the *local generator's capacity*, not cRAG. A bigger model is not automatically better (3b and 7b fail different items), but Answer Relevancy specifically needs a $\geq$7b judge to be trustworthy — the 3b judge false-flags squarely-on-target answers (40/65 vs 63/65 on the *same* answers). The production consumer (Claude Code as host) resists the failure classes the small local models stumble on.

6 Robustness & security

Prompt injection / context poisoning — the RAG primary failure vector — is addressed in two deliberately separated layers. *Structural* (cRAG's job): the data-not-code guarantee is pinned deterministically in `make check` with no model in the loop. *Semantic* (the host's job): cRAG retrieves the poison faithfully into context; resisting it is the retriever-not-judge contract —

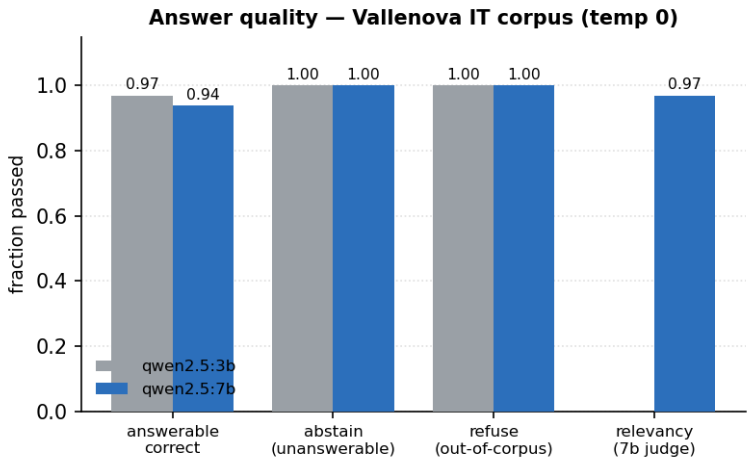


Figure 5: Answer-layer scores. Grounded correctness, abstention on within-corpus-unanswerable questions, and out-of-corpus refusal are near-perfect with *either* local generator. **Answer Relevancy** (does the answer actually address the question?) is the one dimension where the judge model's size matters.

Table 6: RAG-Triad coverage. Context Precision/Recall are covered by the BEIR retrieval metrics above; Faithfulness and Answer Relevancy are scored here by an LLM-as-judge at temp 0.

Triad dimension	Where measured	Result
Context Precision / Recall	BEIR (`make eval`)	nDCG@10 0.74 / Recall@100 0.98 (SciFact)
Faithfulness (answer grounded in retrieved context)	`answers.sh`	near-perfect; hallucination caught when present
Answer Relevancy (answer addresses the question)	`answers.sh`	7b judge 63/65; 3b judge unreliable (40/65)
Correctness (matches hand-written fact)	`answers.sh`	3b 31/32, 7b 30/32 answerable
Abstention / Refusal	`answers.sh`	3/3 abstain, 4/4 out-of-corpus refuse

measured at qwen2.5:3b 2/4 vs qwen2.5:7b 4/4, and the live Claude-Code host resisted the same attack class in a 15-question agent-in-loop run.

7 Operating envelope — scale, memory, and a deliberate scope

A performance report owes an honest answer to "does it scale?". cRAG keeps the full float32 vectors in memory and runs an **exact** cosine scan — no ANN index, no quantization. That is a *deliberate* design point for its target (butlerbot, a single-user personal assistant), not an oversight.

Why exact-and-in-memory is the *right* choice here, not a limitation: at personal scale the cosine scan costs milliseconds and is dwarfed by embedding and generation latency, so there is nothing to optimize; in exchange cRAG is **100% exact** (no recall ceiling beyond the embedder — the fidelity claim of §2 depends on this), dead simple, and supports clean incremental update/delete. Trading those away to solve a scale problem the target does not have would be a poor bargain.

Why not ANN / quantization (yet). These are the standard tools beyond this envelope, and they are *orthogonal*:

- **ANN (HNSW)** fixes *latency* (O(N) → O(log N)) at the cost of exactness (tunable 95–99% recall) — but it does *not* fix memory: the graph sits *on top of* the full vectors (+1.5–2×).
- **Quantization** (int8 / binary / PQ) fixes *memory* (4–32×) at the cost of exactness; int8 is near-lossless, binary needs a full-precision re-rank that erodes the saving.

Table 7: The `make check` gate — every layer passes clean and fully offline.

Layer	What it proves
Unit tests	chunker, cosine math, SQLite store round-trip, JSON, retrieval e2e
Valgrind	zero leaks / invalid reads across the full suite
ASan + UBSan	no heap overflow / UB; fuzzer scratch buffers are red-zoned
TSan	the `--workers N` embed pool is race-free
eval-ci	offline nDCG/Recall/MRR *regression floor* (toy deterministic embedder)
determinism	same query ×100 → byte-identical top-k (6640/6640 assertions)
fuzzer	adversarial bytes never overflow the hand-rolled parsers
injection (structural)	a poisoned chunk reaches the model as escaped **data**, never as prompt structure
4-hour reindex soak	DB self-bounds (873 cycles, 0 drift, ≤1.13× growth); no leak

Table 8: Memory & latency envelope. 1024-dim float32 = ∼4 KB/vector; cosine is an O(N) in-memory scan. Latency at the target scale is dominated by the embedding call and LLM generation, not the scan.

Corpus	Vectors in RAM	Scan / query	Verdict
∼100k chunks (butlerbot)	∼400 MB	a few ms	✓ ideal regime
∼1M chunks	∼4 GB	∼seconds	∼ borderline
20 GB doc base (∼10M chunks)	∼40 GB	minutes	× needs ANN + quant

For billion-scale they are combined (HNSW-PQ, IVF-PQ). Both are on the forward-looking roadmap, gated on a real trigger — a corpus that outgrows RAM, a multi-tenant deployment, or a memory-constrained device — none of which is the butlerbot use case. The conclusion an engineer should take away is not "it doesn't scale" but "it is optimized, on purpose, for the regime where exact search is both feasible and preferable."

8 Methodology & honest caveats

- **What the BEIR comparison licenses.** cRAG's nDCG@10 is its embedder's quality delivered without loss; it is *not* evidence that cRAG's engine beats systems using a different embedder. The only like-for-like engine comparison is BM25 (cRAG-FTS5 vs Anserini), where cRAG is ∼1–3 points behind — the honest engine signal. A rigorous dense parity check (cRAG + embedder X vs the official published number for X) is the natural next measurement.
- **BEIR numbers** are the engine's own `make eval` over the full official query sets, standard nDCG@10 / Recall@k / MRR@10. The grey reference figures are published BEIR points (BM25 Anserini; dense model papers / MTEB) for orientation — full-precision exact search, not re-run here on identical hardware.
- **NFCorpus** is intentionally hard (sparse relevance, medical jargon); low absolute recall there is a property of the corpus.
- **Answer-quality** numbers use *local* ollama generators (qwen2.5:3b/7b) at temp 0 on the Italian corpus — they characterise the pipeline and deliberately expose generator-size effects rather than hide them behind a frontier API.
- The Italian deployment embedder is **bge-m3** (1024-dim, multilingual), chosen on a paraphrase probe (20/20 vs mxbai 8/20); `mxbai-embed-large` is the English benchmark embedder used for the BEIR numbers here.

cRAG — pure C11, single binary, $0 dependencies. All figures reproducible: `make eval DATASET=scifact|nfcorpus`,

`make check`, `bash tests/manual/answers.sh`.